

RABBIT HOLES

Accelerate · Manipal University Jaipur

10

PHASE

Trust

Anyone can commit as you. Signing, releases, and the day a password lands in history.



Contents

1	Commit authorship is a claim, not a fact	3
2	Signing with SSH	3
2.1	Verifying other people locally	4
3	Signing with GPG	4
4	What Verified actually proves	4
5	Tags	5
5.1	Two kinds, and only one is right	5
6	Versioning and releases	6
6.1	Semantic versioning	6
6.2	Releases	6
6.3	Changelogs	6
7	The day someone commits a secret	6
7.1	Prevention	7
8	Identity housekeeping	7
9	Reference	8

1 Commit authorship is a claim, not a fact

In Phase 1 of the workshop you ran this:

```
git config --global user.name "Your Name"
git config --global user.email "you@example.com"
```

Nothing verified either value. Git wrote what you typed into every commit.

TRY IT

In a scratch repository:

```
git config user.name "Linus Torvalds"
git config user.email "torvalds@linux-foundation.org"
echo "hello" > f.txt && git add . && git commit -m "Fix the kernel"
git log
```

Git reports a commit by Linus Torvalds. Push it to a public repository and GitHub will display his name, and if that address is on his account, possibly his avatar. Nothing about this is a bug or an exploit. The author field is a string that git copies from your config.

WHY THIS EXISTS

This is not carelessness in git's design. Git was built for a mailing list workflow where patches arrive by email from strangers and maintainers judge them on content. Identity was never meant to be enforced by the transport.

The consequence is that if you want authorship to mean something, you have to sign.

2 Signing with SSH

Two mechanisms exist, GPG and SSH. GPG came first and is more widely supported. **SSH signing is much easier to set up**, works on GitHub, and reuses the key you already have. Start here.

```
ssh-keygen -t ed25519 -C "you@example.com"
```

Skip that if you already have ~/.ssh/id_ed25519.pub. Then:

```
git config --global gpg.format ssh
git config --global user.signingkey ~/.ssh/id_ed25519.pub
git config --global commit.gpgsign true
git config --global tag.gpgsign true
```

Register the key with GitHub as a **signing** key, which is a separate thing from an authentication key even if it is the same key:

```
gh ssh-key add ~/.ssh/id_ed25519.pub --type signing --title "Signing key"
```

Now every commit is signed automatically. Check:

```
git log --show-signature -1
git verify-commit HEAD
```

2.1 Verifying other people locally

For git to verify someone else's signature it needs to know which keys belong to whom, in an allowed signers file:

```
echo "friend@example.com ssh-ed25519 AAAAC3Nza..." >> ~/.ssh/allowed_signers
git config --global gpg.ssh.allowedSignersFile ~/.ssh/allowed_signers
```

Without it, git can confirm a signature is internally valid but cannot tell you whose it is.

3 Signing with GPG

If your project already uses GPG, or you want signatures verifiable outside GitHub:

```
gpg --full-generate-key
gpg --list-secret-keys --keyid-format=long
```

Take the key ID from the `sec` line and:

```
git config --global user.signingkey 3AA5C34371567BD2
git config --global commit.gpgsign true
gpg --armor --export 3AA5C34371567BD2
gh gpg-key add -
```

TRAP

On Windows and macOS the recurring problem is git being unable to find the agent, or the passphrase prompt failing in a non-interactive terminal. The error is usually `gpg failed to sign the data`.

```
export GPG_TTY=$(tty)
git config --global gpg.program "C:/Program Files (x86)/GnuPG/bin/gpg.exe"
```

Put the `GPG_TTY` line in your shell profile. This single problem is why SSH signing is worth preferring.

4 What Verified actually proves

GitHub shows a green **Verified** badge on signed commits. Be precise about what it means: the commit was signed by a key registered to the account whose email is in the commit. It says nothing about the code being good, reviewed, or safe.

NOTE

There is one twist worth knowing. Commits made through the GitHub web interface, including merging a pull request with the button, are signed by *GitHub's* key and show as verified. So a Verified badge sometimes means “GitHub made this commit” rather than “this human signed it”. Both are legitimate. They are not the same claim.

To require signatures on a branch, use branch protection or a ruleset, covered in Phase 15.

5 Tags

A tag names a commit permanently. Branches move, tags do not.

5.1 Two kinds, and only one is right

```
git tag v1.0.0
```

A **lightweight** tag: a file in `refs/tags` containing a hash, exactly like a branch that never moves.

```
git tag -a v1.0.0 -m "First release"
```

An **annotated** tag: a real object in the database with a tagger, date, message, and optionally a signature.

TRAP

Use annotated tags for anything anyone else will see. Lightweight tags carry no author, no date, and no message, and they cannot be signed. `git describe` ignores them by default, which surprises people whose build version suddenly stops working.

Lightweight tags are fine as a private bookmark. Releases should always be annotated, and ideally signed with `-s`.

```
git tag
git tag -l "v1.*"
git show v1.0.0
git push origin v1.0.0
git push --tags
git tag -d v1.0.0
git push origin --delete v1.0.0
git describe --tags
```

NOTE

Tags are not pushed by `git push`. This catches everyone at least once: you tag a release, push, and the tag is not there. Push it explicitly.

6 Versioning and releases

6.1 Semantic versioning

MAJOR.MINOR.PATCH, and the rules are simple:

MAJOR You broke compatibility. Existing users must change something.

MINOR You added something, and everything that worked before still works.

PATCH You fixed a bug, changed no interface.

The version number is a promise to the people depending on you. Bumping MAJOR is not an admission of failure, it is how you tell people to read the notes before upgrading. Sneaking a breaking change into a PATCH is the actual failure.

6.2 Releases

A GitHub release attaches notes and downloadable files to a tag.

```
gh release create v1.0.0 --title "v1.0.0" --notes "First release"
gh release create v1.0.0 --generate-notes
gh release create v1.0.0 ./dist/app.zip ./dist/app.tar.gz
gh release list
gh release view v1.0.0
gh release download v1.0.0
```

`--generate-notes` writes the notes from merged pull request titles since the last tag. With reasonable PR titles the output is genuinely usable, which is a good argument for insisting on reasonable PR titles.

6.3 Changelogs

CHANGELOG.md exists for humans, in a way that generated release notes are not. The convention worth following groups changes under **Added**, **Changed**, **Deprecated**, **Removed**, **Fixed** and **Security**, newest version at the top.

NOTE

Changelogs are the classic merge conflict, because every pull request appends to the same place, which is exactly the mechanism Phase 3 used deliberately. If it becomes painful, the usual fix is a directory of small files, one per change, combined at release time.

7 The day someone commits a secret

It will happen. An `.env` file, an API key pasted into a config, a private key added by accident.

TRAP

Rotate the credential first. Always, before anything else.

The instinct is to scrub the history immediately, and that instinct is wrong, because it wastes the minutes that matter on the wrong task. Assume the secret is already compromised the moment it is pushed. Public repositories are scraped continuously by bots that exist specifically to find committed keys, and the delay between push and exploitation is routinely measured in minutes.

Revoke and reissue the key. Then clean the history, which is housekeeping.

The order:

1. **Revoke the credential.** Rotate the key, change the password, expire the token.
2. **Check for use.** Look at the provider's access logs.
3. **Remove it from history** with `git filter-repo`, per Phase 5.
4. **Force push, and tell everyone** to re-clone.
5. **Ask GitHub to purge caches**, because commits stay reachable by hash through the API even after a force push. Contact Support for this.
6. **Add it to .gitignore** so it cannot recur.

7.1 Prevention

```
gh secret set API_KEY
gh api -X PATCH repos/:owner/:repo -F security_and_analysis[secret_scanning][status]=enabled
```

GitHub's secret scanning is free on public repositories and detects the token formats of most major providers. **Push protection** goes further and blocks the push before the secret ever lands, which is the only intervention that actually prevents the problem.

Locally, a pre-commit hook that scans staged changes catches it earlier still. Phase 11 covers hooks, and tools like `gitleaks` plug straight into them.

```
gitleaks protect --staged
```

8 Identity housekeeping

Contributors change email addresses, and history ends up with the same person under three names. `.mailmap` fixes reporting without rewriting anything:

```
Vidhu <physicsexplore2023@gmail.com> <old-address@university.edu>
Shashwat Deep <shashwat@example.com> <shashwat@old.example.com>
```

Commit that file at the root and `git shortlog`, `git log` and `git blame` all collapse the duplicates.

9 Reference

Goal	Command
Sign every commit with SSH	<code>git config --global gpg.format ssh</code>
	<code>git config --global commit.gpgsign true</code>
Register a signing key	<code>gh ssh-key add <key> --type signing</code>
Check a signature	<code>git log --show-signature -1</code>
Annotated tag	<code>git tag -a v1.0.0 -m "..."</code>
Signed tag	<code>git tag -s v1.0.0 -m "..."</code>
Push tags	<code>git push --tags</code>
Release with generated notes	<code>gh release create v1.0.0 --generate-notes</code>
Remove a secret from history	<code>git filter-repo --path <f> --invert-paths</code>
Store a secret properly	<code>gh secret set NAME</code>
Merge duplicate identities	<code>.mailmap</code> at the repo root

CHECK YOURSELF

1. Why can you commit under someone else's name, and what stops it mattering?
2. What exactly does GitHub's Verified badge prove?
3. Name two things a lightweight tag cannot do.
4. You pushed an API key ten minutes ago. What is step one, and why is it not rewriting history?
5. Why do tags not go up with a normal `git push`?

Next: Phase 11, *Configuration and automation*, on bending git to your habits: aliases, attributes, and hooks that run your linter before a bad commit can happen.